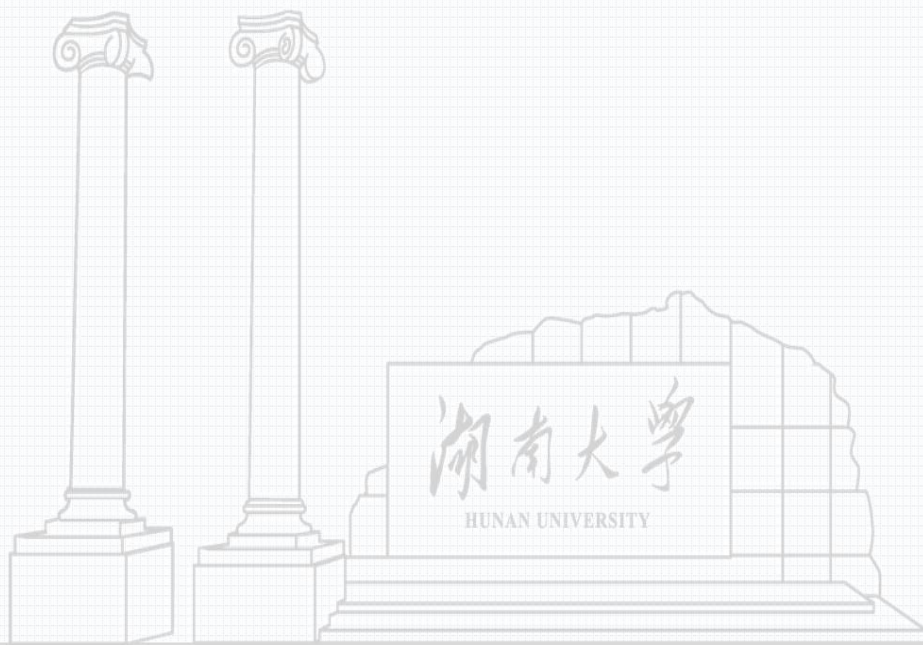


KR (Karp-Rabin) 算法

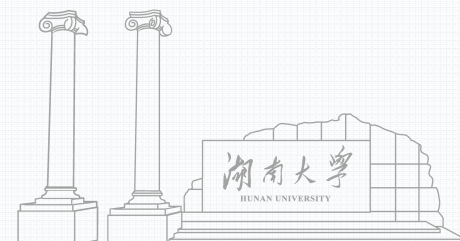
第一次小班课个人资料





目录

- 1 算法介绍
- 2 算法实现步骤
- 3 名词解释
- 4 代码实现
- 5 性能分析
- 6 优缺点与改进方法
- 7 应用场景





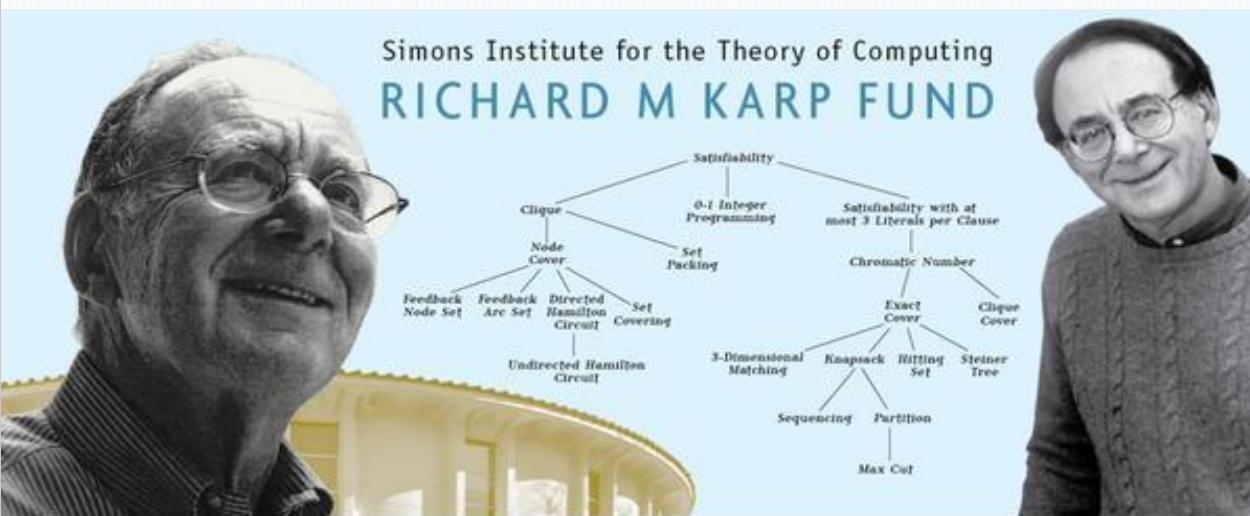
一. 算法介绍

1. 算法的提出

Rabin-Karp算法是由Richard M. Karp和Michael O. Rabin在1987年提出的字符串匹配算法。该算法主要用于在文本中搜索字符串出现的位置。

2. 算法的内容

该算法用于解决以下字符串匹配问题及其一些推广的随机化算法：给定一个长度为 n 的字符串 X （模式串）和一个字符串 Y （目标串），在 Y 中找到 X 作为连续块的首次出现位置。此算法通过将长度为 n 的字符串表示为称为指纹的较短字符串，并通过操作指纹而非较长字符串来实现高效性^[1]



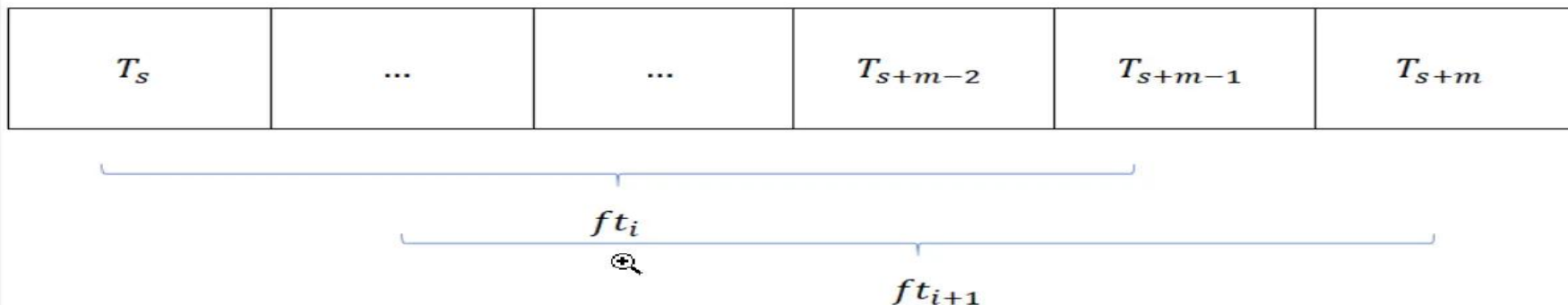


二. 算法实现步骤

Karp-Rabin 算法核心在于使用哈希函数计算出子串的哈希值并通过滚动哈希的方法快速更新哈希值来加速模式串与子串的匹配速率，减少字符逐一比较的次数

算法按以下步骤执行：

1. 计算长度为 m 的模式串的哈希值。
2. 初始化滑动窗口，从主串的第一个位置开始，计算当前窗口（第一个长度为 m 的子串）的哈希值并与模式串的哈希值比较。
3. 如果哈希值相等，为防止哈希碰撞，进行逐字符比较，确认是否完全匹配。
4. 若哈希值不相等或字符串不匹配，将滑动窗口向右移动一个字符，采用滚动哈希的方法更新窗口的哈希值，继续比较直到匹配成功或遍历完整个主串。^[2]





三. 名词解释

- 哈希函数：用于计算字符串的哈希值的函数

字符串是字符数组。然而，字符可以被解释为整数，其确切值取决于所使用的编码类型（例如 ASCII 码）。这意味着我们可以将字符串视为整数数组。找到一种将整数数组转换为单个整数的方法，使我们能够使用期望数字作为输入的哈希函数来哈希字符串。^[2]

$$\text{hash}(S) = (s_0 * b^{m-1} + s_1 * b^{m-2} + \dots + s_{m-1} * b^0) \bmod p$$

字符串 $S = s_0, s_1, \dots, s_{m-1}$ ，其中 $s_0, s_1, \dots, s_{m-1}$ 是字符串的每个字符的数值表示（通常是它们的 ASCII 码或 Unicode 值）。

b ：基数（base），是一个常数值，通常选择一个较大的质数值（如 31 或 256）。选择的基数通常是为了减少碰撞的概率，保证哈希值的均匀分布。

m ：字符串的长度，即模式串字符的个数。

p ：模数（modulus），是一个质数。它帮助将计算结果限制在一个较小的范围内，避免数值溢出。常见的做法是选择一个质数 p ，它大于基数 b ，并且远大于字符串的长度。



三. 名词解释

- 滚动哈希：运用了Rabin指纹字符串的思想快速更新哈希值的方法^[4]

如果独立计算每个子串的哈希值，每次哈希计算都要查看 m 个元素。

若采用滚动哈希计算下一个子串的哈希值只需检查两个元素：离开窗口的元素(s_{left})和进入窗口的元素(s_{right})。这是一个 $O(1)$ 的操作。

在滚动哈希中，当窗口滑动时，新的子串会基于前一个子串的哈希值来更新。通过以下公式，可以在 $O(1)$ 的时间内更新哈希值：

$$\text{new_hash} = (b * (\text{old_hash} - s_{\text{left}} * b^{m-1}) + s_{\text{right}}) \bmod p$$

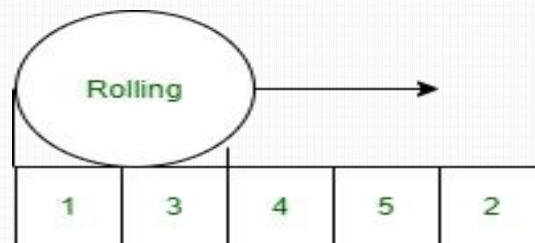
old_hash 是当前窗口的哈希值。

s_{left} 是窗口左侧字符的数值。

s_{right} 是窗口右侧新加入字符的数值。

b 是基数

m 是窗口的大小(模式串的长度)。





三. 名词解释

- 哈希碰撞：它指的是两个不同的输入数据（例如不同的字符串、整数或对象）通过相同的哈希函数计算后得到相同的哈希值。

可能原因：

1. 哈希值的输出范围有限：

哈希函数将一个输入映射到一个固定大小的输出。例如，当我们使用一个 32 位的哈希值时，输出的哈希值范围是有限的，大约为 2^{32} 个不同的哈希值。而在实际应用中，输入数据往往是非常庞大的，如文本、文件或其他类型的数据。假设有 N 个输入数据，而输出的哈希值的空间只有 M 个（如 $M=2^{32}$ ），当 $N>M$ 时，就一定会有两个不同的输入数据映射到相同的哈希值上，即发生冲突。

2. 哈希函数的设计问题：

如果哈希函数设计不好，可能导致某些输入值的哈希值分布不均匀，导致某些哈希值区域过于集中，从而增加冲突的概率。

3. 不具备良好的散列性：

好的哈希函数应该能尽可能地使得每个输入数据映射到不同的哈希值，而不是将大量的输入映射到同一个哈希值。没有良好散列性的哈希函数容易导致冲突。



四. 代码实现

1. 举个例子展示算法实现过程

目标串为abcdef 模式串为bcd

- 首先计算bcd的哈希值

s_m 取各字符的ASCII值: $b=98, c=99, d=100$

假设基数 $b=31$, 这是一种常见的质数选择 (通常选择质数以保证哈希的分布均匀)。

假设 $m=10^9+7$, 这是一个常用的大质数。

计算模式串的哈希值 $\text{hash}(S)=(98*31^2+99*31^1+100*31^0)\bmod(10^9+7)=97207$

- 下面开始计算第一个子串abc的哈希值。采用相同的算法, 得到其哈希值

$\text{hash}(Y1)=96454$

- 窗口向右滑动计算bcd的哈希值

我们需要去除第一个字符 a 对哈希值的贡献, 并加入字符 d。

$\text{Hash}(Y2)=(31*(96454-97*31^2)+100)\bmod(10^9+7)=97207$

- 相同则检查是否每个字符相同, 若都相同, 则匹配成功。

$\text{hash}(S)=(s_0*b^{m-1}+s_1*b^{m-2}+\dots+s_{m-1}*b^0)\bmod p$

$\text{new_hash}=(b*(\text{old_hash}-s_{\text{left}}*b^{m-1})+s_{\text{right}})\bmod p$



四. 代码实现

2. 算法伪代码

RABIN-KARP-MATCHER(T, P, d, q)

$n = T.length$

$m = P.length$

$h = d^{m-1} \bmod q$

$p = 0$

$q = 0$

for $i = 1$ to m

$p = (d_p + P[i]) \bmod q$

$t_0 = (d_{t_0} + T[i]) \bmod q$

for $s = 0$ to $n - m$

 if $p == t_s$

 if $P[1...m] == T[s+1...s+m]$

 if $s < n - m$

$t_{s+1} = (d * (t_s - T[s+1] * h) + T[s+m+1]) \bmod q$



五. 性能分析

时间复杂度：

设目标串长度为 n ，模式串长度为 m

则滚动计算母串哈希值复杂度是 $O(n)$

计算子串哈希值复杂度是 $O(m)$

遍历母串进行哈希值匹配的复杂度是 $O(n)$

综上，Rabin-Karp算法的时间复杂度平均情况是 $O(m+n)$ ，最坏情况 $O(mn)$,当所有窗口都出现哈希碰撞时，最坏情况就会发生。

空间复杂度：

主要用于存储模式串、主串和哈希值，空间复杂度为 $O(1)$ 。

额外的空间开销：需要存储哈希值和部分窗口的数据。

$S(n)$	$O(1)$
$T_{best}(n)$	$O(n)$
$T_{worst}(n)$	$O(nm)$



六. 优缺点与改进方法

优点:

高效性: 相比传统暴力匹配 $O(n * m)$ 的复杂度, 对于大规模数据, 尤其是当模式串长度较短, Karp-Rabin 算法在哈希值不冲突时, 时间复杂度降低为 $O(n + m)$, 大大提高了匹配效率。

简洁性: 算法本身简单, 易于理解和实现。

缺点

哈希值相等时进行逐字符检查, 在哈希冲突较多时, 可能存在性能问题

哈希冲突会影响算法的性能和准确性, 尤其是在哈希函数选择不当时, 时间复杂度可能退化为 $O(n * m)$ 。



六. 优缺点与改进方法

改进方法：

双重哈希：通过使用多个不同的哈希函数（双重哈希）来降低哈希冲突的概率，提高算法的鲁棒性。

选择两个哈希函数：

第一个哈希函数（hash1）：计算模式串的哈希值。

第二个哈希函数（hash2）：通常选择一个与第一个哈希函数完全不同的哈希策略。可以是不同的基数、模数等，计算模式串的另一个哈希值

得到 $\text{hash1}(\text{pattern})$ 和 $\text{hash2}(\text{pattern})$ 。

计算主串每个子串的哈希值：对主串的每个长度为 m 的子串，计算两个哈希值 $\text{hash1}(\text{substring})$ 和 $\text{hash2}(\text{substring})$ 。

如果 $\text{hash1}(\text{substring})$ 和 $\text{hash1}(\text{pattern})$ 相等

并且 $\text{hash2}(\text{substring})$ 和 $\text{hash2}(\text{pattern})$ 也相等，才进行逐字符比较，确认是否匹配。



七. 应用场景

- Karp-Rabin 算法可以在文本编辑器、搜索引擎等中用于快速查找特定子字符串,比如 Semantic Scholar 的文章抄袭检测系统。^[3]
- 数据压缩: 在一些压缩算法中使用 Karp-Rabin 算法进行高效的字符串匹配。
- 病毒检测: 通过匹配病毒特征字符串, 帮助检测计算机病毒。





参考文献

- [1]R. M. Karp and M. O. Rabin, "Efficient randomized pattern-matching algorithms," in IBM Journal of Research and Development, vol. 31, no. 2, pp. 249-260, March 1987
- [2]Massachusetts Institute of Technology, "Rolling Hash (Rabin-Karp Algorithm)"people.csail.mit.edu/alinush, Spring 2011
- [3]<https://zhuanlan.zhihu.com/p/563551141> 2022-09-12
- [4]https://blog.csdn.net/m0_72410588/article/details/131218148 2023-06-14



THANKS!

